Размеры шрифтов
3x5 пикселей - минимальный размер читаемого шрифта
7x12 пикселей - размер шрифта для ascii в старом терминале

🛑**GAP ⇔ Global Average Pooling**
🛑**Transposed Convolution (Deconvolution)**

DWConv ⇔ Depthwise Convolution ⇔ Поканальная свёртка
Обрабатывает каждый канал отдельно

subsampling ⇔ субдискретизация ⇔ уменьшение частоты дискретизации цифрового сигнала или уменьшение разрешения изображения
pooling ⇔ уменьшение пространственной размерности признаков

Receptive Field ⇔ Рецептивное поле ⇔ Область входного изображения, определяющая значение одного признака в последующем слое

Image Gradient ⇔ Градиент изображения
Вектор, указывающий направление наибольшего изменения интенсивности пикселей, координаты которого равны частным производным интенсивности по осям X и Y.

Integral Image ⇔ Интегральное изображение
Двумерный массив, элемент которого по координатам X и Y равен сумме интенсивностей пикселей исходного изображения в прямоугольнике от начала координат до текущей точки.

SSD ⇔ Sum of Squared Differences
Сумма квадратов разностей пикселей двух изображений.

DoG ⇔ Difference of Gaussians
$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$
Изображение, полученное вычитанием двух соседних масштабов масштабного пространства.

Keypoint ⇔ Ключевая точка
Пиксель на изображении, локальная окрестность которого имеет высокую математическую вариативность градиентов по нескольким направлениям.

Descriptor ⇔ Дескриптор
N-мерный вектор чисел, описывающий локальную структуру окрестности ключевой точки для вычисления метрики расстояния.

LRN ⇔ Local Response Normalization

0. Метод нормализации, подавляющий активность соседних нейронов при высокой активации текущего. Конкретно: нормализация активаций в окне n по каналам.
0.1. был вытеснен BN

1. $$b^i_{x,y} = \frac{a^i_{x,y}}{\left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a^j_{x,y})^2\right)^\beta}$$
2. $b^i_{x,y}$ — нормализованное значение.
3. $a^i_{x,y}$ — выходное значение до нормализации от фильтра i в пространственной позиции x, y.
4. N — общее количество фильтров в слое.
5. n — количество соседних фильтров.
6. k, α, β — гиперпараметры нормализации.

V-SLAM ⇔ Visual Simultaneous Localization and Mapping
Оценка траектории камеры и построение 3D-карты в реальном времени.

SfM ⇔ Structure from Motion
Построение 3D-геометрии сцены по набору неупорядоченных фотографий.

DCN ⇔ Deformable Convolutional Networks
Деформируемые сверточные сети — свертки, пространственные смещения которых обучаются нейросетью.

Affine transformation ⇔ Аффинное преобразование
0. геометрическое преобразование, которое переводит точки, прямые и плоскости одного пространства в другое, строго сохраняя три главных свойства:
1. Прямолинейность: прямые линии всегда остаются прямыми.
2. Параллельность: параллельные линии всегда остаются параллельными.
3. Пропорциональность: сохраняется отношение длин отрезков, лежащих на одной прямой.

Box-фильтр ⇔ Box Filter
Ядро свертки, состоящее из одинаковых значений, вычисляющее среднее арифметическое пикселей в прямоугольной области.

SIFT ⇔ Scale-Invariant Feature Transform ⇔ Масштабно-инвариантное преобразование признаков
0. Метод поиска ключевых точек и построения 128-мерных вещественных дескрипторов, инвариантных к масштабу и повороту.
1. Детектор точек: поиск локальных экстремумов в трехмерном тензоре DoG по координатам X, Y и масштабу σ. Точка сравнивается с 26 соседними пикселями: 8 в текущем масштабе и по 9 в верхнем и нижнем масштабах.

2. Локализация: применение разложения Тейлора 2-го порядка для определения субпиксельных координат экстремума.

$$D(x) = D + \frac{\partial D^T}{\partial x}x + \frac{1}{2}x^T\frac{\partial^2 D}{\partial x^2}x$$

где $x -$ вектор смещения от центра пикселя, $D -$ значение функции в точке, производные вычисляются по конечным разностям.

3. Фильтрация контуров: отсечение точек с большим главным радиусом кривизны и малым перпендикулярным радиусом.

$$\frac{\mathrm{Tr}(H)^2}{\mathrm{Det}(H)} < \frac{(r+1)^2}{r}$$

где $\mathrm{Tr}(H) -$ след матрицы Гессе H, $\mathrm{Det}(H) -$ определитель матрицы Гессе, $r = 10 -$ пороговое отношение большего собственного числа к меньшему.

4. Ориентация:

$$m(x,y) = \sqrt{(L_{x+1,y} - L_{x-1,y})^2 + (L_{x,y+1} - L_{x,y-1})^2}$$

$$\theta(x,y) = \mathrm{atan2}(L_{x,y+1} - L_{x,y-1}, L_{x+1,y} - L_{x-1,y})$$

где $L -$ интенсивность пикселя в соответствующем масштабе, $m(x,y) -$ модуль градиента, $\theta(x,y) -$ угол градиента. Строится гистограмма из 36 бинов по 10 градусов каждая. Максимум гистограммы задает базовый угол θ.

5. Построение дескриптора:

5.1. Окрестность 16×16 пикселей вокруг точки ориентируется по базовому углу θ.

5.2. Окрестность делится на 4×4 подобласти по 4×4 пикселя каждая.

5.3. В каждой подобласти строится гистограмма градиентов на 8 направлений.

5.4. Вектор признаков состоит из $4 \times 4 \times 8 = 128$ значений.

6. Нормализация дескриптора: вектор делится на свою L2-норму. Значения вектора ограничиваются сверху порогом 0.2, после чего вектор нормализуется повторно.

SURF ⇔ Speeded Up Robust Features ⇔ Ускоренные устойчивые признаки

0. Метод, аппроксимирующий вычисления SIFT с помощью матриц Гессе и интегральных изображений для уменьшения размерности вектора и снижения вычислительной сложности.

1. Детектор точек: поиск экстремумов определителя аппроксимированной матрицы Гессе.

2. Аппроксимация: вторые производные гауссиана заменяются на Box-фильтры.

$$\mathrm{Det}(H_{approx}) = D_{xx}D_{yy} - (0.9D_{xy})^2$$

где $D_{xx}, D_{yy}, D_{xy} -$ отклики Box-фильтров, аппроксимирующие производные, $0.9 -$ весовой коэффициент компенсации погрешности аппроксимации.

3. Формирование масштабного пространства: масштаб σ увеличивается расширением размера Box-фильтров от 9×9 до 15×15 и 21×21. Вычисляется за $O(1)$ операций через интегральное изображение.

4. Ориентация: вычисление откликов вейвлетов Хаара по осям X и Y в круговой окрестности радиуса $6s$, где s — масштаб точки. Отклики суммируются в скользящем секторе с углом $\frac{\pi}{3}$. Вектор с максимальной длиной задает ориентацию.
5. Построение дескриптора:
5.1. Квадратная область размером $20s \times 20s$ выравнивается по ориентации и делится на 4×4 подобласти.
5.2. В каждой подобласти вычисляются 4 суммы откликов вейвлетов Хаара:
$v = (\sum dx, \sum dy, \sum |dx|, \sum |dy|)$
где dx, dy — отклики вейвлетов по осям X и Y.
5.3. Вектор признаков состоит из $4 \times 4 \times 4 = 64$ значений.
6. Вектор нормализуется по L2-норме.

ORB ⇔ Oriented FAST and Rotated BRIEF ⇔ Ориентированный FAST и повернутый BRIEF
0. Метод построения бинарных дескрипторов, инвариантных к повороту. Работает на два порядка быстрее SIFT.
1. oFAST ⇔ Oriented FAST — детектор углов: пиксель p считается углом, если на окружности радиуса 3 пикселя, содержащей 16 пикселей, существует $N = 9$ подряд идущих пикселей x, для которых выполняется условие $I_x > I_p + t$ или $I_x < I_p - t$, где t — порог интенсивности. Точки фильтруются по мере Харриса на всех уровнях пирамиды изображений.
2. Ориентация: вычисляется через моменты интенсивности.
$m_{pq} = \sum_{x,y} x^p y^q I(x, y)$
где m_{pq} — пространственный момент интенсивности порядка p и q, x, y — координаты пикселя, $I(x, y)$ — интенсивность.
$C = (\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}})$
где C — центр масс окрестности.
$\theta = \text{atan2}(m_{01}, m_{10})$
где θ — угол поворота окрестности.
3. rBRIEF ⇔ Rotated BRIEF — бинарный дескриптор длины 256 бит, занимающий 32 байта памяти.
3.1. Задается набор из 256 пар пикселей A_i и B_i в окрестности точки размером 31×31 пиксель.
3.2. Координаты пар поворачиваются на угол θ умножением на матрицу поворота R_θ.
3.3. Бит i равен 1, если $I(A_i) < I(B_i)$, иначе 0.
4. Обучение паттерна пар: 256 пар точек выбраны из 100000 случайных вариантов по жадному алгоритму, максимизирующему дисперсию каждого бита и минимизирующему корреляцию битов между собой.
5. Сравнение дескрипторов ORB осуществляется через расстояние Хэмминга аппаратно через операцию XOR и подсчет единичных битов.

HOG ⇔ Histogram of Oriented Gradients ⇔ Гистограмма направленных градиентов
0. Метод плотного описания формы объектов через статистическое распределение градиентов интенсивностей в регулярной сетке без выделения ключевых точек.
1. Вычисление градиентов: применение одномерных ядер свертки $[-1, 0, 1]$ по оси X и $[-1, 0, 1]^T$ по оси Y.
$m(x, y) = \sqrt{dx^2 + dy^2}$
$\theta(x, y) = \text{atan2}(dy, dx)$
где dx, dy — отклики свертки по осям, $m(x, y)$ — модуль градиента, $\theta(x, y)$ — угол градиента.
2. Разбиение на ячейки: изображение делится на ячейки размером 8×8 пикселей.
3. Построение гистограмм: внутри ячейки градиенты распределяются по гистограмме из 9 корзин. Каждая корзина охватывает 20 градусов в диапазоне от 0 до 180 градусов без учета знака градиента. В корзину прибавляется значение модуля градиента $m(x, y)$.
4. Разбиение на блоки: смежные ячейки объединяются в блоки размером 2×2 ячейки. Блок содержит 16×16 пикселей. Перекрытие соседних блоков составляет 1 ячейку. Вектор блока v имеет размерность $2 \times 2 \times 9 = 36$.
5. Нормализация блоков:
$$v_{norm} = \frac{v}{\sqrt{||v||_2^2 + \epsilon^2}}$$
где v — исходный вектор блока, $||v||_2^2$ — квадрат L2-нормы вектора, $\epsilon = 10^{-5}$ — константа предотвращения деления на ноль.
6. Итоговый дескриптор: конкатенация векторов всех нормализованных блоков окна поиска. Для классического окна 64×128 пикселей, вмещающего 7×15 блоков, размерность равна $7 \times 15 \times 36 = 3780$ признаков.

Prewitt Operator ⇔ Оператор Прюитта
0. Метод вычисления дискретной аппроксимации первой производной интенсивности изображения.
0.1. K_x, K_y ⇔ Ядра свертки оператора Прюитта.
1. Предпосылки: Необходимость аппаратно-эффективного вычисления градиента с одновременным усреднением значений в ортогональном направлении для подавления ВЧ-флуктуаций.
2. Формулы вычисления компонент градиента:
$G_x = K_x * I$
$G_y = K_y * I$
2.1. G_x — горизонтальный градиент (выделяет вертикальные границы).
2.2. G_y — вертикальный градиент (выделяет горизонтальные границы).
3. Матрицы ядер свертки Прюитта 3×3:

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$K_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

3.1. Ядро K_x вычисляет разность интенсивностей правого и левого столбцов пикселей, игнорируя центральный столбец. Строки складываются с равным весом 1.

4. Формула модуля градиента:

$$|G| = \sqrt{G_x^2 + G_y^2}$$

4.1. Вычислительно оптимальная аппроксимация для систем с ограниченными ресурсами: $|G| \approx |G_x| + |G_y|$.

5. Формула направления вектора градиента:

$$\theta = \arctan(\frac{G_y}{G_x})$$

Sobel Operator ⇔ Оператор Собеля

0. Метод аппроксимации градиента, модифицирующий оператор Прюитта за счет введения дифференцированных весовых коэффициентов для центральных пикселей с целью улучшения изотропии (независимости от направления).

0.1. S_x, S_y ⇔ Ядра свертки оператора Собеля.

1. Предпосылки: Оператор Прюитта присваивает одинаковый вес всем пикселям в строке или столбце, что приводит к искажениям при диагональных перепадах яркости. Собель ввел весовой коэффициент 2 для центральной оси, аппроксимируя функцию Гаусса.

2. Матрицы ядер свертки Собеля 3×3:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$S_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

3. Вычисление G_x, G_y, $|G|$ и θ производится по формулам, идентичным оператору Прюитта, с заменой матриц K_x и K_y на S_x и S_y.

4. Использование коэффициента 2 увеличивает вес центрального пикселя, что снижает чувствительность алгоритма к локальным ВЧ-флуктуациям по сравнению с Прюиттом.

Canny Edge Detector ⇔ Алгоритм Канни

0. Многоступенчатый алгоритм обнаружения границ, математически оптимизирующий три критерия: минимизацию ложных срабатываний, максимизацию точности локализации границы и обеспечение единственного отклика на одну физическую границу.

0.1. NMS ⇔ Non-Maximum Suppression ⇔ Подавление немаксимумов.
0.2. T_{high} ⇔ Верхний порог интенсивности градиента.
0.3. T_{low} ⇔ Нижний порог интенсивности градиента.
0.4. σ ⇔ Стандартное отклонение функции Гаусса.
0.5. $H(x, y)$ ⇔ Двумерная функция Гаусса.
1. Этап 1: Гауссово сглаживание. Изображение I сворачивается с двумерным фильтром Гаусса для устранения ВЧ-флуктуаций.
2. Формула ядра Гаусса:
$$H(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$
2.1. $x, y -$ смещение пикселя относительно центра ядра свертки.
2.2. Увеличение значения σ приводит к более сильному размытию, скрывая мелкие детали и оставляя только крупные контуры.
3. Этап 2: Вычисление градиентов. Изображение обрабатывается оператором Собеля для нахождения модулей $|G|$ и направлений θ.
3.1. Непрерывные значения угла θ принудительно округляются до ближайшего из четырех направлений: 0°, 45°, 90°, 135°.
4. Этап 3: NMS. Процесс утончения границ до ширины ровно в один пиксель.
4.1. Алгоритм сканирует матрицу $|G|$. Для каждого пикселя проверяются два соседних пикселя, лежащих на линии, заданной углом θ.
4.2. Если значение $|G|$ текущего пикселя строго больше значений обоих соседей, оно сохраняется. В противном случае $|G|$ приравнивается к 0.
5. Этап 4: Double Thresholding ⇔ Двойное пороговое преобразование. Классификация оставшихся пикселей на три группы.
5.1. Если $|G| \geq T_{high} -$ пиксель маркируется как сильная граница.
5.2. Если $|G| < T_{low} -$ пиксель маркируется как фон (значение 0).
5.3. Если $T_{low} \leq |G| < T_{high} -$ пиксель маркируется как слабая граница.
6. Этап 5: Edge Tracking by Hysteresis ⇔ Трассировка границ с гистерезисом.
6.1. Пиксель со статусом слабой границы переводится в статус сильной границы только в том случае, если среди его 8 смежных соседей присутствует минимум один пиксель, являющийся сильной границей.
6.2. Все оставшиеся слабые границы приравниваются к 0.




Парадигмы решения задач классификации и локализации объектов:

1. Two-Stage and Multi-Stage Detectors ⇔ Region-based ⇔ Двухстадийные и многостадийные детекторы

1.1. External Proposal-based ⇔ С внешним эвристическим выделением регионов (R-CNN, Fast R-CNN, SPP-Net)

1.2. Internal RPN-based ⇔ С внутренним нейросетевым выделением регионов (Faster R-CNN, Mask R-CNN, Light-Head R-CNN)

1.3. Multi-Stage Cascade ⇔ Многостадийные каскадные детекторы (Cascade R-CNN, Cascade Mask R-CNN, Hybrid Task Cascade)

2. Single-Stage Detectors ⇔ One-stage ⇔ Одностадийные детекторы

2.1. One-stage Anchor-based ⇔ Одностадийные на основе шаблонов

2.1.1. Single-Scale Grid ⇔ Одномасштабные с фиксированной сеткой (YOLOv2, SqueezeDet, OverFeat)

2.1.2. Multi-Scale without Fusion ⇔ Мультимасштабные без слияния признаков (SSD, MultiBox, MS-CNN)

2.1.3. Multi-Scale FPN-based ⇔ Мультимасштабные со слиянием признаков (RetinaNet, YOLOv3, YOLOv4)

2.2. One-stage Anchor-free ⇔ Одностадийные безанкорные

2.2.1. Single-Scale Grid Anchor-free ⇔ Одномасштабные безанкорные с фиксированной сеткой (YOLOv1, DenseBox, UnitBox)[1][2]

2.2.2. Dense Grid/Pixel Regression ⇔ Плотная мультимасштабная попиксельная/сеточная регрессия (FCOS, YOLOv8, YOLOX)

2.2.3. Keypoint-Based ⇔ На основе ключевых точек (CornerNet, ExtremeNet, CenterNet-Triplets)

3. Query-based Transformers ⇔ На основе глобального внимания

3.1. Pure End-to-End / Bipartite Matching ⇔ С двудольным сопоставлением без NMS (DETR, Deformable DETR, Conditional DETR)

3.2. Hybrid / Real-Time Transformers ⇔ Гибридные трансформеры реального времени (RT-DETR, DINO-DETR, Co-DETR)

4. Diffusion-based detectors ⇔ Диффузионные детекторы

4.1. Direct Bounding Box Denoising ⇔ Прямое удаление случайных отклонений из координат рамок (DiffusionDet, DiffuDETR, GenDet)

5. Open-vocabulary & Generative VLMs ⇔ Мультимодальные детекторы с открытым словарем и LVLM

5.1. Contrastive-based Open-Vocabulary ⇔ На основе выравнивания контрастивных признаков (GLIP, OWL-ViT, Grounding DINO)

5.2. Generative/Autoregressive MLLMs ⇔ Авторегрессионные генеративные мультимодальные модели (Kosmos-2, CogVLM, Qwen-VL)

6. 3D Object Detection ⇔ Трехмерная локализация

6.1. LiDAR-only 3D Detectors ⇔ На основе лидарных облаков точек (PointPillars, SECOND, PV-RCNN)
6.2. Camera-based 3D / BEV Detectors ⇔ На основе монокулярных/мультиспективных камер (FCOS3D, DETR3D, BEVFormer)
6.3. Multi-modal Fusion 3D Detectors ⇔ Мультимодальный синтез LiDAR + Camera (MVX-Net, TransFusion, BEVFusion)
6.4. Radar-involved 3D Detectors ⇔ Детекторы с интеграцией радаров (CenterPoint-Radar, CRAFT, RC-BEVFusion)

1. Two-Stage and Multi-Stage Detectors ⇔ Region-based ⇔ Двухстадийные и многостадийные детекторы
[Input Image] -> [Backbone] -> [Region Proposal Network (RPN)] -> [ROI Pooling] -> [Fully Connected Layers] -> [BBox/Class]

RoI ⇔ Region of Interest
Часть изображения (не обязательно прямоугольник), в которой может быть объект. Bounding Box - ответ, RoI - предположение.

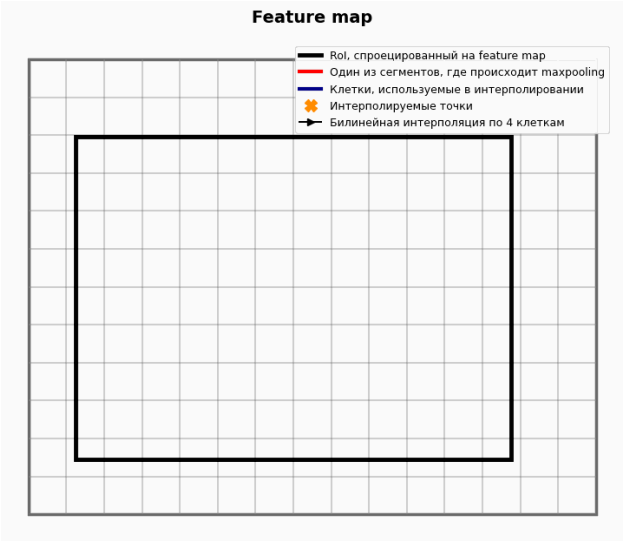
Selective search
Алгоритм выделения RoI, не нейросеть
1. Разбиение на суперпиксели методом Фельзеншвальба - начальные регионы
2.1 Вычисляется сходство S между регионами:
$S(r_i, r_j) = \alpha_1 S_{color} + \alpha_2 S_{texture} + \alpha_3 S_{size} + \alpha_4 S_{fill}$
S_{color} - пересечение гистограмм цветов (для каждого региона строится гистограмма 75 бинов, 25 на один канал)
$S_{texture}$ - сходство гистограмм, каждая из 240 значений - градиенты в 8 направлениях для каждого канала
$$S_{size} = 1 - \frac{size(r_i) + size(r_j)}{size(image)}$$
$$S_{fill}(r_i, r_j) = 1 - \frac{size(BB_{ij}) - size(r_i) - size(r_j)}{size(image)}$$
size(BB_{ij}) - площадь минимального BoundingBox который вмещает одновременно r_i, r_j
2.2 Регионы с максимальным S объединяются в новый r_{new}
2.3 Добавляется BoundingBox который описывает r_{new} в список RoI
2.4 удаляются старые r_i, r_j

R-CNN ⇔ Region-based Convolutional Neural Network
1. Selective search - Предсказываются ~2000 потенциальных областей
2. Каждая область приводится к разрешению 227x227
3. Каждая область проходит CNN блок
4. На выходе CNN - вектор

5. На векторах обучается SVM (тогда практика показывала что он лучше чем softmax слой на 3-4% mAP)

RoI Pooling
Унифицирование размерности RoI через maxpooling с округлениями на feature map. Нужен для MLP.
1. Предсказываются координаты RoI на исходном изображении
2. Координаты проецируются на feature map, получается область размерами HxW, H, W непрерывные
3. Полученная область делится на части с округлением, происходит maxpooling
4. Результат maxpooling всегда имеет заданные размерности

RoI Align
Унифицирование размерности RoI через maxpooling с интерполяцией на feature map. Нужен для MLP.
1. Предсказываются координаты RoI на исходном изображении
2. Координаты проецируются на feature map, получается область размерами HxW, H, W непрерывные
3. Полученная область виртуально делится на части
3.1 В каждой части билинейно интерполируются P точек. Обычно их 4, и расположены равноудаленно от друг друга и краев части. Интерполяция происходит по 4 ближайшим клеткам к интерполируемой точке.
3.2 Maxpooling для интерполированных точек внутри части
4. Результат maxpooling всегда имеет заданные размерности



Fast R-CNN
1. Параллельно:
1.1 Вся картинка проходит предобученный CNN
1.2 Selective search - Выделяются ROI (BBox) на исходном изображении

2. ROI Pooling
3. Классификация + регрессия координат

Anchor boxes
Заранее заданные эталонные рамки разных размеров и соотношений сторон
Идут наборами по k штук
Обычно используется 3 масштаба ($128^2, 256^2, 512^2$) и 3 соотношения сторон (1:1, 1:2, 2:1) (k=9 рамок в итоге)
Где располагаются: на исходном изображении. Центр набора anchor boxes приставлен к каждому пикселю карты активации, этот пиксель проецируется на исходное изображение, вокруг него размещаются anchor boxes.

RPN ⇔ Region Proposal Network
1.0 Для каждого пикселя feature map HxW подразумевается k anchor boxes. Проецируя пиксель feature map на исходное изображение как центр, можно представить вокруг него k anchor boxes. Разница в геометрии anchor boxes не будет учитываться на этапе свертки, но предсказания будут делаться подразумевая, что они все разные. По feature map сначала идет свертка 3x3 с выходом HxWxd, потом две свертки 1x1xd.
1.1 cls layer: По карте активации скользит маленькая сверточная (1x1xd, выход HxWx2k) сеть-классификатор, она определяет, есть ли объект или нет (дает два числа на anchor - foreground score, background score).
1.2 reg layer: По карте активации скользит маленькая сверточная (1x1xd, выход HxWx4k) сеть-регрессор, которая показывает, как изменить координаты каждого anchor boxes (tx,ty,tw,th)
(сети 1.1 и 1.2 работают параллельно)
2. Обрабатываются выходы двух малых сверточных сетей (без обучения, эвристики)
2.0 anchor boxes масштабируются под предсказания reg layer (1.2)
2.1 Упорядочиваются по уверенности предсказания (cls score)
2.2 NMS ⇔ Non-Maximum Suppression ⇔ Удаляются рамки с IoU выше заданного порога (обычно 0.7), оставляя одну с наибольшим confidence score
2.3 Выбор top-N рамок
Multi-Task Loss RPN:

$$L(\{p_i\},\{t_i\}) = \frac{1}{N_{cls}}\sum_i L_{cls}(p_i, p_i^*) + \lambda\frac{1}{N_{reg}}\sum_i p_i^* L_{reg}(t_i, t_i^*)$$

x,y - координаты центра ограничивающей рамки
w,h - ширина и высота ограничивающей рамки
t_x, t_y - Безразмерные смещения центра относительно размеров анкора

$$t_x = (x - x_a)/w_a, \quad t_y = (y - y_a)/h_a$$
$$t_x^* = (x^* - x_a)/w_a, \quad t_y^* = (y^* - y_a)/h_a$$

t_w, T_h - Логарифмическое масштабирование размеров

$$t_w = \log(w/w_a), \quad t_h = \log(h/h_a)$$

$$t_w^* = \log(w^*/w_a), \quad t_h^* = \log(h^*/h_a)$$
$$L_{reg}(t_i, t_i^*) = \sum_{j \in \{x,y,w,h\}} \text{smooth}_{L1}(t_{i,j} - t_{i,j}^*)$$
$$\text{smooth}_{L1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

Faster R-CNN
1. Картинка проходит backbone, вынимаем feature maps последнего слоя CNN блока
2. RPN на feature map из (1)
3. RoI Pooling / RoI Align с координатами из (2) на feature map из (1)
4. Детектор (MLP) на унифицированных векторах из (3) -> Ответ

2. Single-Stage Detectors ⇔ One-stage ⇔ Одностадийные детекторы
[Input Image] -> [Backbone] -> [Detection Head (Сверточные слои)] -> [BBox/Class]
Решает задачи классификации и локализации объектов за один forward pass без промежуточного этапа генерации предложений областей.
В отличие от multi-stage detectors, где RPN отбрасывает фоновый класс, single-stage приходится разбираться с ним на уровне предсказания нейросетью - экстремальный дисбаланс классов. Для этого придуман focal loss.

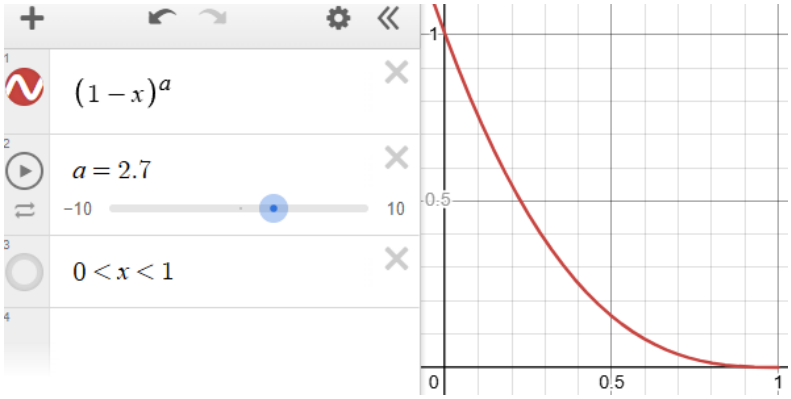
IoU ⇔ Intersection over Union
$$\text{IoU} = \frac{\text{Area}(B_p \cap B_{gt})}{\text{Area}(B_p \cup B_{gt})}$$
Box
prediction
ground **t**ruth

GIoU ⇔ Generalized Intersection over Union
$$GIoU = IoU - \frac{|B_{all} \setminus (B_p \cup B_{gt})|}{|B_{all}|}$$
Box, **p**rediction, **g**round **t**ruth
B_{all}- минимальная рамка, включающая B_p и B_{gt}

mAP ⇔ mean Average Precision
$$mAP = \frac{1}{N} \sum_{i=1}^{N} AP_i$$
AP ⇔ Average Precision ⇔ площадь под PR-кривой

mAP@0.5: Расчет при пороге IoU = 0.5
mAP@[.5:.95]: Среднее значение mAP, рассчитанное при разных порогах IoU (от 0.5 до 0.95 с шагом 0.05) (наиболее строгий стандарт оценки (COCO mAP))

Focal Loss
$FL(p_t) = -\alpha_t(1-p_t)^{\gamma}log(p_t)$
p_t - уверенность в предсказании (если 1 то p, если 0 то 1-p)
γ варьирует, как уменьшается для примеров с высоким p_t ошибка:

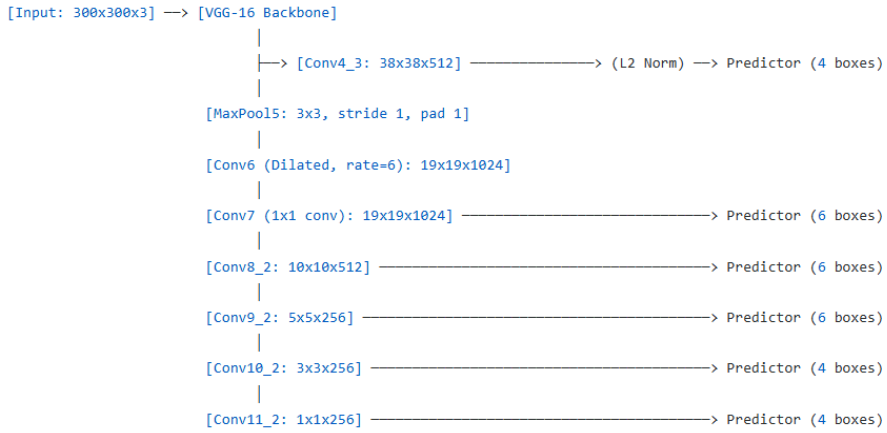


(хочется, чтобы ошибка для классов, которые предсказываются с маленькой уверенностью, была больше)
α_t заменяет в формуле истинную метку класса y: для объекта (y=1) $\alpha_t = \alpha$, для фона (y=0) $\alpha_t = 1 - \alpha$. Он нужен для смягчения дисбаланса классов (фона больше чем объектов)

2.1. One-stage Anchor-based ⇔ Одностадийные на основе шаблонов
На всё изображение накладывается сетка, и каждый узел рассматривается как центр анкоров. Нейросеть предсказывает смещения координат центра и изменение размеров anchor box.

SSD ⇔ Single Shot MultiBox Detector
0. Cl - количество классов, H_i,W_i - размеры feature map
1. CNN Backbone проходит картинка фиксированного размера
2. Создание Default Boxes
2.1. Выбираются feature maps работы backbone над картинкой (1)
2.2. На выбранных feature maps каждому пикселю (тензору 1x1xd) создаются K default boxes (K для каждого feature map разный)
3. Составление предсказаний
3.1 На каждом feature map для каждого default box составляет предсказания предиктор - две параллельные сверточные сети 3x3, одна на регрессию H_ixW_ix(K*4) - смещение координат и изменение размеров, другая на классификацию H_ixW_ix(K*$(Cl + 1_{\text{фон}})$)
3.2. Координаты предсказаний (3.1) переводятся в единый масштаб, все предсказания объединяются в глобальный тензор.

3.3 (inference) Для каждого класса проводится NMS по этому тензору
3.3. (train) hard-negative mining: Предсказания класса background выбираются в соотношении 3:1 background:объект по невозрастанию confidence loss (для уменьшения дисбаланса классов)



Identity Bottleneck

0. Входящий тензор X_0 размерности HxWxC

1. $X_1 = Act(BN(Conv2D_{1\times1\times C_r}(X_0)))$, выход: $H \times W \times C_r$ (сжатие глубины)

2. $X_2 = Act(BN(Conv2D_{3\times3\times C_r}(X_1, padding = same)))$, выход: $H \times W \times C_r$ (обработка)

3. $X_3 = BN(Conv2D_{1\times1\times C}(X_2))$, выход: $H \times W \times C_{out}$ (восстановление глубины)

4. $Y = Act(X_3 + X_0)$ (residual connection)

BN - BatchNorm

$C_{out} = r \cdot C_r = C$

Projection Bottleneck ⇔ Downsampling Bottleneck

0. Входящий тензор X_0 размерности HxWxC; (Resnet v1.5)

1.1. $X_1 = Act(BN(Conv2D_{1\times1\times C_r}(X_0, stride = 1)))$, выход: $H \times W \times C_r$ (сжатие глубины)

1.2. $X_2 = Act(BN(Conv2D_{3\times3\times C_r}(X_1, stride = 2, padding = 1)))$, выход: $\frac{H}{2} \times \frac{W}{2} \times C_r$ (обработка)

1.3. $X_3 = BN(Conv2D_{1\times1\times C_{out}}(X_2))$, выход: $\frac{H}{2} \times \frac{W}{2} \times C_{out}$ (восстановление глубины)

2. $X_{shortcut} = BN(Conv2D_{1\times1\times C_{out}}(X_0, stride = 2))$

3. $Y = Act(X_3 + X_{shortcut})$ (skip connection)

BN - BatchNorm

$C_{out} = r \cdot C_r$; В ResNet-50: $C_{out} = 4C$

CSP ⇔ Cross-Stage Partial Network

1. Разделение по глубине пополам на X_1, X_2 (оригинальная статья)
2. X_1 обрабатывается $X_1 \rightarrow X'_1$; X_2 остается неизменным
3. X'_1, X_2 конкатенируются обратно в X'
4. $Y = Act(BN(Conv2D_{1\times1\times C_{out}}(X')))$

C3 ⇔ CSP YOLOv5

1. Тензор проходит две свёртки $1 \times 1 \times C_{mid}$ параллельно, получаются X_1, X_2
2. X_1 обрабатывается подряд идущими $n \times Bottleneck$; X_2 остается неизменным
3. $X' = concat[X_{bot_n}, X_2]$
4. $Y = Act(BN(Conv2D_{1\times1\times C_{out}}(X')))$

C2f ⇔ CSP Bottleneck with 2 convolutions, fast ⇔ CSP YOLOv8+

0. X размерности $H \times W \times C$, $C_{mid} = \frac{C}{2}$
1. $[X_1, X_2] = Split(Conv2D_{1\times1\times2C_{mid}}(X)),\ X_1, X_2 \in R^{H\times W\times C_{mid}}$
2.1. X_2 обрабатывается подряд идущими $n \times Bottleneck$; X_1 остается неизменным
2.2. Все выходы Bottleneck $X_{bot_1}, X_{bot_2}, \dots, X_{bot_n}$ сохраняются
3. $X' = concat[X_1, X_2, X_{bot_1}, \dots, X_{bot_n}]$
4. $Y = Act(BN(Conv2D_{1\times1\times C_{out}}(X')))$

🛑**C3k**

C3k2

1. $[X_1, X_2] = Split(Conv2D_{1\times1\times2C_{mid}}(X)),\ X_1, X_2 \in R^{H\times W\times C_{mid}}$
2.1. X_2 обрабатывается $n \times Bottleneck_k, k - kernel\ size$ второго фильтра блока ; X_1 остается неизменным
2.2. Все выходы Bottleneck $X_{bot_1}, X_{bot_2}, \dots, X_{bot_n}$ сохраняются
3. $X' = concat[X_1, X_2, X_{bot_1}, \dots, X_{bot_n}]$
4. $Y = Act(BN(Conv2D_{1\times1\times C_{out}}(X')))$

PSABlock ⇔ Position-Sensitive Attention block

0. $X \in R^{B\times C\times H\times W}$; h - количество голов внимания; $d_v = \frac{C}{h}$ размерность V; $d_k = \frac{d_v}{2} = \frac{C}{2h}$;

DWConv - Depthwise convolution

1. $Q, K, V = Split(Conv2D_{1\times1\times(h\cdot(2d_k+d_v))}(X))$; $Q, K \in R^{B\times h\times d_k\times(H\cdot W)}, V \in R^{B\times h\times d_v\times(H\cdot W)}$

2. Параллельно:

2.1. $Attn = V \times (Softmax(\frac{Q^T K}{\sqrt{d_k}}))^T$

2.2. $V' = DWConv_{3\times3}(V.reshape(B \times C \times H \times W))$

3. $X_{Attn} = X + Conv_{1\times1}(Attn.reshape(B \times C \times H \times W) + V')$ (residual connection)

4. $Y = X_{Attn} + Conv_{1\times1}(Act(Conv_{1\times1}(X_{Attn}))$ (residual connection)

SPPF

0. На вход X

1. 3 MaxPool 5x5 padding same подряд, выходы Y_1, Y_2, Y_3

2. Конкатенация: $out = [X, Y_1, Y_2, Y_3]$

SPP ⇔ Spatial Pyramid Pooling Fast

0. $X_0 -$ входной тензор

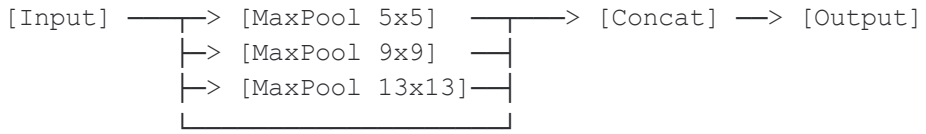
1. Параллельно:

1.1. $X1 = Maxpool_{5\times5}(X_0, padding = same)$

1.2. $X2 = Maxpool_{9\times9}(X_1, padding = same)$

1.2. $X3 = Maxpool_{13\times13}(X_2, padding = same)$

2. $Y = concat[X_0, X_1, X_2, X_3]$



C2PSA ⇔ Cross-Stage Partial Position-Sensitive Attention

C2 ⇔ 2 Convolutions ⇔ Потому что две свёртки в начале и в конце блока

0. X размерности $B \times C \times H \times W$, $C_{mid} = C \cdot e, e = \frac{1}{2}$

1. $[X_1, X_2] = Split(Conv2D_{1\times1\times2C_{mid}}(X))$, $X_1, X_2 \in R^{B\times C_{mid}\times H\times W}$

2. $X_{2out} = X_2$ проходит n подряд блоков PSA

3. $Y = Conv_{1\times1}(\ concat(X_1, X_{2out})\ )$

Stem stage

0. Входящий тензор X_0 размерности HxWxC

1. $Y = Maxpool_{3\times3}(Act(BN(Conv2D_{7\times7\times64}(X, stride = 2, padding = 3))), stride = 2)$

FPN ⇔ Feature Pyramid Network

0. Определения

0.1. $X \in R^{3\times H_{img}\times W_{img}}$

0.2. IB ⇔ Identity Bottleneck, PB ⇔ Projection Bottleneck

0.3. IB^n⇔$IB(IB(\cdots)\ \cdots)$; $PB(X, stride = k)$⇔ PB, где у $Conv2D_{3\times3}(\cdots, stride = k)$ (шаг 1.2 PB) и $X_{shortcut} = BN(Conv2D_{1\times1}(\cdots, stride = k))$ (шаг 2 PB)

0.4. Bottom up ⇔ $H\downarrow$, $W\downarrow, C\uparrow$; Top-Down ⇔ $H\uparrow$, $W\uparrow, C\downarrow$

1. Bottom->up

1.1.1. $C_1 = Stem(X)$, $C_1 \in R^{64\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

1.1.2. P'_1 нет

1.2.1. $C_2 = IB^2(PB(C_1, stride = 1)))$, $C_2 \in R^{256\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

1.2.2. $P'_2 = Conv2D_{1\times1\times256}(C_2)$, $P'_2 \in R^{256\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

1.3.1. $C_3 = IB^3(PB(C_2, stride = 2)))$. $C_3 \in R^{512\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

1.3.2. $P'_3 = Conv2D_{1\times1\times256}(C_3), P'_3 \in R^{256\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

1.4.1. $C_4 = IB^5(PB(C_3, stride = 2)))$, $C_4 \in R^{1024\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

1.4.2. $P'_4 = Conv2D_{1\times1\times256}(C_4)$, $P'_4 \in R^{256\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

1.5.1. $C_5 = IB^2(PB(C_4, stride = 2)))$, $C_5 \in R^{2048\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

1.5.2. $P'_5 = Conv2D_{1\times1\times256}(C_5)$, $P'_5 \in R^{256\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

2. Top->Down (Up->Bottom)

2.1.1. $M_5 = P'_5, M_5 \in R^{256\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

2.1.2. $P_5 = Conv2D_{3\times3\times256}(M_5, stride = 1, padding = 1)$, $P_5 \in R^{256\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

2.2.1. $M_4 = Upsample_{2\times}(M_5) + P'_4, M_4 \in R^{256\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

2.2.2 $P_4 = Conv2D_{3\times3\times256}(M_4, stride = 1, padding = 1)$, $P_4 \in R^{256\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

2.3.1. $M_3 = Upsample_{2\times}(M_4) + P'_3, M_3 \in R^{256\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

2.3.2. $P_3 = Conv2D_{3\times3\times256}(M_3, stride = 1, padding = 1),\ P_3 \in R^{256\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

2.4.1. $M_2 = Upsample_{2\times}(M_3) + P'_2, M_2 \in R^{256\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

2.4.2. $P_2 = Conv2D_{3\times3\times256}(M_2, stride = 1, padding = 1),\ P_2 \in R^{256\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

3. $Y = P_2$

PANet ⇔ Path Aggregation Network

0. Определения

0.1. $X \in R^{3\times H_{img}\times W_{img}}$

0.2. IB ⇔ Identity Bottleneck, PB ⇔ Projection Bottleneck

0.3. $IB^n \Leftrightarrow IB(IB(\cdots)\ \cdots);\ PB(X, stride = k) \Leftrightarrow$ PB, где у $Conv2D_{3\times3}(\cdots, stride = k)$ (шаг 1.2 PB) и $X_{shortcut} = BN(Conv2D_{1\times1}(\cdots, stride = k))$ (шаг 2 PB)

0.4. Bottom up ⇔ $H\downarrow,\ W\downarrow, C\uparrow$; Top-Down ⇔ $H\uparrow,\ W\uparrow, C\downarrow$

1. Bottom->up

1.1.1. $C_1 = Stem(X),\ C_1 \in R^{64\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

1.1.2. P'_1 нет

1.2.1. $C_2 = IB^2(\ PB(C_1,\ stride = 1))),\ C_2 \in R^{256\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

1.2.2. $P'_2 = Conv2D_{1\times1\times256}(C_2),\ P'_2 \in R^{256\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

1.3.1. $C_3 = IB^3(\ PB(C_2,\ stride = 2))).\ C_3 \in R^{512\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

1.3.2. $P'_3 = Conv2D_{1\times1\times256}(C_3)\ , P'_3 \in R^{256\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

1.4.1. $C_4 = IB^5(\ PB(C_3,\ stride = 2))),\ C_4 \in R^{1024\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

1.4.2. $P'_4 = Conv2D_{1\times1\times256}(C_4),\ \ P'_4 \in R^{256\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

1.5.1. $C_5 = IB^2(\ PB(C_4,\ stride = 2))),\ C_5 \in R^{2048\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

1.5.2. $P'_5 = Conv2D_{1\times1\times256}(C_5),\ P'_5 \in R^{256\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

2. Top->Down (Up->Bottom)

2.1.1. $M_5 = P'_5, M_5 \in R^{256\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

2.1.2. $P_5 = Conv2D_{3\times3\times256}(M_5, stride = 1, padding = 1),\ P_5 \in R^{256\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

2.2.1. $M_4 = Upsample_{2\times}(M_5) + P'_4, M_4 \in R^{256\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

2.2.2 $P_4 = Conv2D_{3\times3\times256}(M_4, stride = 1, padding = 1),\ P_4 \in R^{256\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

2.3.1. $M_3 = Upsample_{2\times}(M_4) + P'_3, M_3 \in R^{256\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

2.3.2. $P_3 = Conv2D_{3\times3\times256}(M_3, stride = 1, padding = 1),\ P_3 \in R^{256\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

2.4.1. $M_2 = Upsample_{2\times}(M_3) + P'_2, M_2 \in R^{256\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

2.4.2. $P_2 = Conv2D_{3\times3\times256}(M_2, stride = 1, padding = 1),\ P_2 \in R^{256\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

3. Bottom->up

3.1.1. $N_2 = P_2 \in R^{256\times\frac{H_{img}}{4}\times\frac{W_{img}}{4}}$

3.1.2. $N'_2 = Act(BN(Conv2D_{3\times3\times256}(N_2, stride = 2, padding = 1))), N'_2 \in R^{256\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

3.2.1. $N''_3 = N'_2 + P_3$

3.2.2. $N_3 = Act(BN(Conv2D_{3\times3\times256}(N''_3, stride = 1, padding = 1))),\ N_3 \in R^{256\times\frac{H_{img}}{8}\times\frac{W_{img}}{8}}$

3.2.3. $N'_3 = Act(BN(Conv2D_{3\times3\times256}(N_3, stride = 2, padding = 1))),\ N'_3 \in R^{256\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

3.3.1. $N''_4 = N'_3 + P_4, N''_4 \in R^{256\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

3.3.2. $N_4 = Act(BN(Conv2D_{3\times3\times256}(N''_4, stride = 1, padding = 1))),\ N_4 \in R^{256\times\frac{H_{img}}{16}\times\frac{W_{img}}{16}}$

3.3.3. $N'_4 = Act(BN(Conv2D_{3\times3\times256}(N_3, stride = 2, padding = 1))),\ N'_4 \in R^{256\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

3.4.1. $N''_5 = N'_4 + P_5,\ N''_5 \in R^{256\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

3.4.2. $N_5 = Act(BN(Conv2D_{3\times3\times256}(N''_5), stride = 1, padding = 1))),\ N_5 \in R^{256\times\frac{H_{img}}{32}\times\frac{W_{img}}{32}}$

4. $Y = \{N_2, N_3, N_4, N_5\}$

- YOLO (v1-v10, YOLOX)

- RetinaNet
- ProgLoss
- STAL
- MuSGD

Feature Pyramid

feature maps с разных слоев работы CNN.

🛑FCOS, CenterNet
🛑FCN (Fully Convolutional Network), DeepLab, Atrous Spatial Pyramid Pooling (ASPP)
🛑Mask R-CNN
🛑Panoptic FPN
🛑DCGAN, WGAN, WGAN-GP (Gradient Penalty), StyleGAN (v1-v3), cGAN, Pix2Pix, CycleGAN
🛑DDPM (Denoising Diffusion Probabilistic Models), DDIM
🛑Stable Diffusion (1.5, XL, 3), Classifier-Free Guidance
🛑Vision Transformer (ViT), Patch embedding, Swin Transformer (Shifted window), DETR (DEtection TRansformer), Bipartite matching loss, Segment Anything Model (SAM).
🛑LVLM
🛑
🛑
🛑
🛑

LeNet Subsampling

$y = \sigma(w \cdot (\frac{x_1+x_2+x_3+x_4}{4}) + b)$

w,b - обучаемые
пулинг с масштабированием с обучаемыми параметрами

LeNet-5

0. Архитектура CNN для классификации изображений размером 32x32 пикселя по 10 классам. Архитектура минимизирует количество параметров за счет общих весов в ядрах свертки и использования неполного графа связей между слоями.
0.1.1. C-layer ⇔ Convolutional layer ⇔ Сверточный слой
0.1.2. S-layer ⇔ Subsampling layer ⇔ Слой подвыборки
0.1.3. F-layer ⇔ Fully connected layer ⇔ Полносвязный слой
0.2. Оригинально определялись числа 0-9 по центрированному изображению числа 32x32. Для loss использовалось евклидово расстояние между 10 трафаретами чисел и выходом сети, которое имело такие же пространственные размерности.
0.3. Вместо max pooling, которого тогда не было, использовались обучаемые пулинги lenet subsampling, потому что обычный average pooling сильно ухудшал качество, размывая изображение
1. Сеть содержит 7 вычисляющих слоев. Входной слой в нумерации не учитывается. Обозначения слоев содержат букву типа слоя и порядковый номер.
2. В слоях C1, S2, C3, S4, C5, F6 применяется функция активации Scaled Tanh.

2.1. Формула вычисления: $f(a) = A\tanh(S \cdot a)$.
2.2. $A = 1.7159$.
2.3. $S = \frac{2}{3}$.
2.4. При заданных константах выполняется условие $f(1) = 1$ и $f(-1) = -1$. Вторая производная функции достигает максимума при $a = 1$.
3. Input: Входной тензор размерности 32x32x1. Содержит 1 канал.
3.1. Пиксели фона нормализованы до значения -0.1.
3.2. Пиксели объектов нормализованы до значения 1.175.
4. C1: C-layer.
4.1. Вход: 32x32x1.
4.2. Параметры свертки: 6 ядер размера 5x5, Stride = 1, Padding = 0.
4.3. Выход: 6 Feature Map размера 28x28.
4.4. Формула вычисления количества параметров: $(H_{kernel} \cdot W_{kernel} \cdot C_{in} + 1) \cdot C_{out}$.
4.5. Количество параметров: $(5 \cdot 5 \cdot 1 + 1) \cdot 6 = 156$.
4.6. Количество связей: $156 \cdot 28 \cdot 28 = 122304$.
5. S2: S-layer.
5.1. Вход: 28x28x6.
5.2. Размер окна подвыборки: 2x2, Stride = 2, Padding = 0.
5.3. Выход: 6 Feature Map размера 14x14.
5.4. В слое применяется операция LeNet Subsampling с последующей передачей результата в функцию Scaled Tanh.
5.5. Каждая Feature Map имеет 1 обучаемый вес и 1 обучаемое смещение.
5.6. Количество параметров: $(1 + 1) \cdot 6 = 12$.
5.7. Количество связей: $(4 + 1) \cdot 6 \cdot 14 \cdot 14 = 5880$.
6. C3: C-layer с неполным графом связей.
6.1. Вход: 14x14x6.
6.2. Параметры свертки: 16 ядер размера 5x5, Stride = 1, Padding = 0.
6.3. Выход: 16 Feature Map размера 10x10.
6.4. Карты признаков слоя C3 соединены с картами слоя S2 по асимметричной матрице для нарушения симметрии сети и сокращения числа вычислений.
6.4.1. Карты 0, 1, 2, 3, 4, 5 слоя C3 соединены с 3 последовательными картами S2.
6.4.2. Карты 6, 7, 8, 9, 10, 11 слоя C3 соединены с 4 последовательными картами S2.
6.4.3. Карты 12, 13, 14 слоя C3 соединены с 4 непоследовательными картами S2.
6.4.4. Карта 15 слоя C3 соединена со всеми 6 картами S2.
6.5. Количество параметров:
$6 \cdot (5 \cdot 5 \cdot 3 + 1) + 9 \cdot (5 \cdot 5 \cdot 4 + 1) + 1 \cdot (5 \cdot 5 \cdot 6 + 1) = 456 + 909 + 151 = 1516$.
6.6. Количество связей: $1516 \cdot 10 \cdot 10 = 151600$.
7. S4: S-layer.
7.1. Вход: 10x10x16.
7.2. Размер окна подвыборки: 2x2, Stride = 2, Padding = 0.
7.3. Выход: 16 Feature Map размера 5x5.

7.4. В слое применяется операция LeNet Subsampling.
7.5. Количество параметров: $(1 + 1) \cdot 16 = 32$.
7.6. Количество связей: $(4 + 1) \cdot 16 \cdot 5 \cdot 5 = 2000$.
8. C5: C-layer.
8.1. Вход: 5x5x16.
8.2. Параметры свертки: 120 ядер размера 5x5, Stride = 1, Padding = 0.
8.3. Выход: 120 Feature Map размера 1x1. Размер ядер совпадает с пространственным размером входа, поэтому математически данный слой эквивалентен F-layer.
8.4. Количество параметров: $(5 \cdot 5 \cdot 16 + 1) \cdot 120 = 48120$.
8.5. Количество связей равно числу параметров: 48120.
9. F6: F-layer.
9.1. Вход: 120 нейронов.
9.2. Выход: 84 нейрона. Размерность 84 соответствует размеру растрового представления ASCII-символов 7x12 пикселей.
9.3. Количество параметров: $(120 + 1) \cdot 84 = 10164$.
10. Output: Выходной слой на основе RBF.
10.1. Вход: 84 нейрона.
10.2. Выход: 10 нейронов, соответствующие цифрам от 0 до 9.
10.3. Выходное значение y_i для i-го класса вычисляется как квадрат евклидова расстояния: $y_i = \sum_{j=0}^{83} (x_j - w_{ij})^2$.
10.4. x_j — выход j-го нейрона слоя F6.
10.5. w_{ij} — параметр i-го RBF-нейрона, представляющий j-й пиксель эталонного изображения класса. Веса w_{ij} фиксированы и не обновляются при обучении.
10.6. Количество параметров: $84 \cdot 10 = 840$.
11. Loss Function вычисляется по формуле: $E = \frac{1}{P} \sum_{p=1}^{P} y_{d_p}(Z^p)$.
11.1. P — общее количество примеров в обучающей выборке.
11.2. y_{d_p} — выход RBF-нейрона, соответствующего истинному классу d_p для входного тензора Z^p.
11.3. В качестве алгоритма оптимизации применяется метод градиентного спуска с диагональной аппроксимацией матрицы Гессе.

AlexNet
0. Состоит из 8 слоев: 5 слоев Conv и 3 слоев FC.
1. Входной тензор имеет размерность $227 \times 227 \times 3$.
1.1. 227 — высота и ширина в пикселях.
1.2. В оригинальной статье указан размер 224×224, однако математически корректным для получения тензора 55×55 после Conv1 является размер 227×227, так как $(227 - 11)/4 + 1 = 55$.

1.3. 3 — количество каналов RGB.
2. Conv1: 96 фильтров размера 11×11, Stride 4, Padding 0.
2.1. Активация: ReLU.
2.2. Пулинг: Max Pooling с окном 3×3 и Stride 2.
2.3. Нормализация: LRN с параметрами $k = 2, n = 5, \alpha = 10^{-4}, \beta = 0.75$.
3. Conv2: 256 фильтров размера 5×5, Stride 1, Padding 2.
3.1. Активация: ReLU.
3.2. Пулинг: Max Pooling с окном 3×3 и Stride 2.
3.3. Нормализация: LRN с параметрами $k = 2, n = 5, \alpha = 10^{-4}, \beta = 0.75$.
4. Conv3: 384 фильтра размера 3×3, Stride 1, Padding 1.
4.1. Активация: ReLU.
5. Conv4: 384 фильтра размера 3×3, Stride 1, Padding 1.
5.1. Активация: ReLU.
6. Conv5: 256 фильтров размера 3×3, Stride 1, Padding 1.
6.1. Активация: ReLU.
6.2. Пулинг: Max Pooling с окном 3×3 и Stride 2.
7. FC6: 4096 нейронов.
7.1. Активация: ReLU.
7.2. Регуляризация: Dropout с вероятностью 0.5.
8. FC7: 4096 нейронов.
8.1. Активация: ReLU.
8.2. Регуляризация: Dropout с вероятностью 0.5.
9. FC8: 1000 нейронов.
9.1. Активация: Softmax.

Архитектурные и алгоритмические особенности
0. Математические и инженерные решения, снижающие ошибку классификации.
1. ReLU: Ускоряет достижение 25% ошибки обучения в 6 раз по сравнению с Tanh.
2. Перекрывающийся пулинг (Overlapping Pooling):
2.1. Использование окна субдискретизации большего размера, чем Stride (размер 3, Stride 2).
2.2. Снижает ошибку на 0.4% по сравнению с неперекрывающимся пулингом (окно 2, Stride 2).
3. Разделение на два GPU:
3.1. Фильтры слоев распределены между двумя GPU поровну для обхода аппаратного ограничения видеопамяти.
3.2. Коммуникация между GPU происходит только в слоях Conv3, FC6, FC7, FC8. Выходы Conv1 и Conv2 на первом GPU не передаются на второй GPU.
4. Data Augmentation:
4.1. Трансляция: Извлечение случайных фрагментов 227×227 из исходных изображений 256×256.
4.2. Горизонтальное отражение: Применяется к извлеченным фрагментам.
4.3. Цветовая модификация:

4.3.1. $I_{xy}^{new} = I_{xy} + [p_1, p_2, p_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T$
4.3.2. I_{xy} — исходный вектор пикселя.
4.3.3. p_i — собственный вектор i-й главной компоненты из ковариационной матрицы RGB пикселей.
4.3.4. λ_i — собственное значение i-й главной компоненты.
4.3.5. α_i — случайная величина из нормального распределения со средним 0 и стандартным отклонением 0.1.

VGG-16 ⇔ Visual Geometry Group - 16
0. Модель CNN, состоящая из 16 слоев с обучаемыми весами: 13 слоев Conv и 3 слоев FC.
0.1. Все, что в блоках, выполняется последовательно: 4.1->4.2->4.3->5.1->...
0.2. В каждой свёртке padding=same, stride=1
0.3 Общее количество обучаемых параметров составляет 138 357 544.
1. Блок 1:
1.1. Conv 3x3, 64 канала, ReLU.
1.2. Conv 3x3, 64 канала, ReLU.
1.3. Max Pooling.
1.4. Выходной тензор имеет размерность $112 \times 112 \times 64$.
2. Блок 2:
2.1. Conv 3x3, 128 каналов, ReLU.
2.2. Conv 3x3, 128 каналов, ReLU.
2.3. Max Pooling.
2.4. Выходной тензор имеет размерность $56 \times 56 \times 128$.
3. Блок 3:
3.1. Conv 3x3, 256 каналов, ReLU.
3.2. Conv 3x3, 256 каналов, ReLU.
3.3. Conv 3x3, 256 каналов, ReLU.
3.4. Max Pooling.
3.5. Выходной тензор имеет размерность $28 \times 28 \times 256$.
4. Блок 4:
4.1. Conv 3x3, 512 каналов, ReLU.
4.2. Conv 3x3, 512 каналов, ReLU.
4.3. Conv 3x3, 512 каналов, ReLU.
4.4. Max Pooling.
4.5. Выходной тензор имеет размерность $14 \times 14 \times 512$.
5. Блок 5:
5.1. Conv 3x3, 512 каналов, ReLU.
5.2. Conv 3x3, 512 каналов, ReLU.
5.3. Conv 3x3, 512 каналов, ReLU.
5.4. Max Pooling.
5.5. Выходной тензор имеет размерность $7 \times 7 \times 512$.
6. Блок 6:

6.1. Развертывание тензора $7 \times 7 \times 512$ в одномерный вектор длиной 25 088 элементов.
6.2. FC слой на 4096 нейронов, ReLU, Dropout с вероятностью $p = 0.5$.
6.3. FC слой на 4096 нейронов, ReLU, Dropout с вероятностью $p = 0.5$.
6.4. FC слой на 1000 нейронов.
7. Выходной слой вычисляет Softmax для 1000 классов ImageNet.

Особенности архитектуры VGG
0. Метод построения сверточных нейронных сетей на основе использования исключительно малых фильтров свертки размером 3x3 и последовательного увеличения глубины сети.
1. Унификация параметров свертки: во всех слоях Conv размер фильтра равен 3x3, Stride равен 1, Padding равен 1.
2. Унификация параметров субдискретизации: во всех слоях Max Pooling размер окна равен 2x2, Stride равен 2.
3. Достижение эквивалентности рецептивных полей при снижении вычислительной сложности: последовательное применение нескольких фильтров 3x3 заменяет один фильтр большего размера.
4. Формула вычисления количества параметров для слоев с равным количеством входных и выходных каналов C: $P = N \times (K^2 \times C^2)$, где N – количество слоев, K – размер фильтра.
5. Замена одного слоя Conv 7x7 тремя слоями Conv 3x3 уменьшает количество весов. Параметры для одного слоя 7x7 равны $49C^2$. Параметры для трех слоев 3x3 равны $3 \times (9C^2) = 27C^2$.
6. Внедрение трех слоев 3x3 вместо одного 7x7 добавляет три функции активации ReLU вместо одной, увеличивая количество нелинейных преобразований.
7. Входные данные подаются в сеть в виде тензора размерности $224 \times 224 \times 3$. На этапе предобработки из каждого цветового канала вычитается среднее значение, вычисленное по всему обучающему набору данных.

VGG-19 ⇔ Visual Geometry Group - 19
0. Модель CNN, состоящая из 19 слоев с обучаемыми весами: 16 слоев Conv и 3 слоев FC. Отличается от VGG-16 добавлением одного дополнительного слоя Conv в блоки 3, 4 и 5.
0.1. Общее количество обучаемых параметров составляет 143 667 240.
1. Блок 1: 2 слоя Conv 3x3, 64 канала, разделенные ReLU, с последующим Max Pooling.
2. Блок 2: 2 слоя Conv 3x3, 128 каналов, разделенные ReLU, с последующим Max Pooling.
3. Блок 3: 4 слоя Conv 3x3, 256 каналов, разделенные ReLU, с последующим Max Pooling.
4. Блок 4: 4 слоя Conv 3x3, 512 каналов, разделенные ReLU, с последующим Max Pooling.
5. Блок 5: 4 слоя Conv 3x3, 512 каналов, разделенные ReLU, с последующим Max Pooling.
6. Блок 6:
6.1. Развертывание тензора $7 \times 7 \times 512$ в одномерный вектор длиной 25 088 элементов.
6.2. FC слой на 4096 нейронов, ReLU, Dropout с вероятностью $p = 0.5$.
6.3. FC слой на 4096 нейронов, ReLU, Dropout с вероятностью $p = 0.5$.

6.4. FC слой на 1000 нейронов.
7. Выходной слой вычисляет Softmax для 1000 классов ImageNet.

🛑Inception (v1-v4)

ResNet (Residual connections, Bottleneck layers)
0. Архитектура CNN для решения Degradation Problem посредством введения Shortcut Connections. Это позволяет обучать сети глубиной более 100 слоев.
1. Базовое уравнение Residual Block имеет вид $y = \mathcal{F}(x, \{W_i\}) + x$.
1.1. x — входной тензор.
1.2. y — выходной тензор.
1.3. $\mathcal{F}(x, \{W_i\})$ — остаточная функция, подлежащая обучению.
2. Для блока из двух слоев формула раскрывается как $\mathcal{F} = W_2\sigma(W_1x)$. σ обозначает функцию ReLU. Смещения слоев опущены из-за применения BN.
3. При несовпадении размерностей тензоров x и $\mathcal{F}$ применяется линейная проекция W_s посредством свертки 1×1. Формула принимает вид $y = \mathcal{F}(x, \{W_i\}) + W_sx$. Несовпадение размерностей возникает при изменении количества каналов или использовании Stride больше 1.
4. Функция активации ReLU применяется после выполнения сложения. Итоговый выход блока равен $\sigma(y)$.
5. Использование Identity Mapping предполагает, что если оптимальное преобразование близко к тождественному, веса внутренних слоев $\mathcal{F}$ стремятся к нулю при обучении.

Внутренняя структура блоков
0. Построение сети осуществляется на основе двух типов блоков: Basic Block и Projection Bottleneck.
1. Basic Block применяется в сетях ResNet-18 и ResNet-34.
1.1. Состоит из двух сверточных слоев 3×3.
1.2. Последовательность операций: свертка 3×3, BN, ReLU, свертка 3×3, BN, сложение с тензором x, ReLU.
2. Projection Bottleneck применяется в сетях ResNet-50, ResNet-101 и ResNet-152.
2.1. Состоит из трех сверточных слоев: 1×1, 3×3 и 1×1.
2.2. Первая свертка 1×1 уменьшает количество каналов Feature Map.
2.3. Свертка 3×3 обрабатывает данные с уменьшенной размерностью.
2.4. Вторая свертка 1×1 восстанавливает исходное количество каналов.
2.5. Вычислительная конфигурация снижает количество обучаемых параметров и FLOPs по сравнению с двумя слоями 3×3.

Особенности архитектуры

0. Дополнительные архитектурные решения, обеспечивающие сходимость модели.
1. Слой BN применяется строго после каждой свертки и перед функцией активации ReLU. При наличии линейной проекции W_s в Shortcut Connection, после нее также применяется BN перед сложением.
2. Модель не использует слои Dropout. Регуляризация сети достигается за счет применения BN.
3. Использование GAP вместо операции Flatten перед FC слоем исключает необходимость хранения матрицы весов для пространственного агрегирования тензора.
4. Все веса сверточных и полносвязных слоев инициализируются с использованием He Initialization.

ResNet-50
0. Общая структура состоит из начального сверточного слоя, пулинга, 4 стадий остаточных блоков, GAP и FC. Входной тензор имеет размерность $224 \times 224 \times 3$.
1. Conv1. Свертка 7×7, 64 фильтра, Stride 2, Padding 3. Размерность выходного тензора $112 \times 112 \times 64$.
2. Max Pooling. Окно пулинга 3×3, Stride 2, Padding 1. Размерность выходного тензора $56 \times 56 \times 64$.
3. Conv2_x. Содержит 3 Projection Bottlenecks. Каждый блок включает свертки 1×1 на 64 фильтра, 3×3 на 64 фильтра, 1×1 на 256 фильтров. Размерность выходного тензора $56 \times 56 \times 256$.
4. Conv3_x. Содержит 4 Projection Bottlenecks. Каждый блок включает свертки 1×1 на 128 фильтров, 3×3 на 128 фильтров, 1×1 на 512 фильтров. Первый блок применяет Stride 2. Размерность выходного тензора $28 \times 28 \times 512$.
5. Conv4_x. Содержит 6 Projection Bottlenecks. Каждый блок включает свертки 1×1 на 256 фильтров, 3×3 на 256 фильтров, 1×1 на 1024 фильтра. Первый блок применяет Stride 2. Размерность выходного тензора $14 \times 14 \times 1024$.
6. Conv5_x. Содержит 3 Projection Bottlenecks. Каждый блок включает свертки 1×1 на 512 фильтров, 3×3 на 512 фильтров, 1×1 на 2048 фильтров. Первый блок применяет Stride 2. Размерность выходного тензора $7 \times 7 \times 2048$.
7. GAP. Усредняет пространственные размерности. Размерность выходного тензора $1 \times 1 \times 2048$.
8. FC. Линейный слой, преобразующий вектор размерности 2048 в вектор размерности 1000 для классификации ImageNet. К выходу применяется Softmax.
9. ResNet-50 содержит 25.5 миллионов параметров и требует выполнения 3.8 миллиардов FLOPs для прямого прохода одного изображения.

🛑DenseNet, Depthwise Separable Convolution (MobileNet), EfficientNet (Compound scaling), ConvNeXt.
🛑CRNN